

Best Practice for AI Agents Project

Chapter 5

Routing, Orchestration, and Multi-Agent Teamwork

Junxian Zhu¹ Yiran Sun² Guanping Dai³

LinkMind Project

¹ Junxian Zhu, LinkMind Project Team, [LandingBJ](#)

² Yiran Sun, LinkMind Project Team, [Xiamen University Malaysia](#)

³ Guanping Dai, LinkMind Project Team, Founder and CEO of [LandingBJ](#)

Contents

About the Contributors	2
Chapter 5. Routing, Orchestration, and Multi-Agent Teamwork	3
Learning Objectives	4
5.1 Routing as a First-Class Decision Layer	4
5.2 Orchestration Is the Work Path, Not the Model Path	5
5.3 Multi-Agent Design as Responsibility Architecture	6
5.4 The Topology of Multiple Lobsters, One LinkMind	6
5.5 Hands-On Lab: A Multi-Agent Incident Workflow	7
5.6 Observability, Cost, and Reliability in Teamed Agent Systems . .	10
5.7 The Team Operating Model	10
5.8 Common Anti-Patterns	11
5.9 Collaboration Patterns beyond the Simple Trio	11
5.10 Distributed Topologies: Proxy, GEO, Cascade, and Local Acceleration	12
5.11 Measuring Maturity in Multi-Agent Programs	13
5.12 Adoption Roadmap and Organizational Design	13
Chapter Summary	14
Review Questions	15
Further Study: Shared Standards and Cross-Team Budgeting	15
Further Study: Migration from Isolated Successes to a Common Plane	16

About the Contributors

Junxian Zhu

LinkMind Project Team, [LandingBJ](#)

- Focus on the research and development of large model products, covering algorithms and applications
- Rich experience in full-process development and delivery of government and enterprise projects

Yiran Sun

LinkMind Project Team, [Xiamen University Malaysia](#)

- Focus on AI agent engineering practices and applied AI workflows
- Background in computer science, machine learning, and AI-related system research

Guanping Dai

LinkMind Project Team, Founder and CEO of [LandingBJ](#)

- Over 20 years of experience in AI algorithm research and development. Built proprietary deep learning and large-model frameworks
- Extensive experience in banking and telecommunications system architecture. Participated in image analysis projects for the National Automotive Safety Laboratory and telemetry tracking systems for Shenzhou spacecraft recovery capsules
- Former experience at the Chinese Academy of Sciences Software Institute
- Former experience at BEA/Oracle (China)

5

CHAPTER

Routing, Orchestration, and Multi-Agent Teamwork

Agent systems become strategically important when they stop being isolated assistants and start becoming a coordinated layer of work. At that point three questions emerge at once. Which model or backend should handle each request? In what sequence should reasoning, retrieval, and tool calls occur? And when several agents participate, which agent owns which responsibility? The first question is routing, the second is orchestration, and the third is multi-agent design. Confusing them leads to muddled architecture and vague accountability.

The final chapter brings these questions together because LinkMind is especially revealing when several agents share one substrate. The platform materials describe routing strategies such as failover, polling, parallel invocation, best-route selection, and pass-through compatibility. They also describe topologies such as Agent Server, Agent Mate, Security Gateway, Proxy plus GEO plus Cascade, and multi-team shared middle layers. Read as engineering patterns, these ideas tell a coherent story: many agent surfaces can remain diverse as long as one control plane governs model access, capabilities, knowledge boundaries, cost policy, and audit.

This chapter also returns to the user's requested scenario directly: multiple Lobster agents, one LinkMind. Hermes Agent enters as a natural counterpart for structured review or workflow orchestration. The objective is not to celebrate a particular interface. The objective is to show how several interfaces and agent roles can cooperate without fragmenting the platform underneath them.

Learning Objectives

- Differentiate routing, orchestration, and multi-agent responsibility design.
- Understand why model choice should be policy-driven rather than hard-coded per client.
- Use LinkMind patterns to centralize cost, failover, observability, and safety for several agents at once.
- Build and inspect a multi-agent workflow with distinct roles and shared capabilities.
- Translate the topology of “multiple Lobsters, one LinkMind” into a sustainable team operating model.

5.1 Routing as a First-Class Decision Layer

Routing answers a specific question: given this task, which backend path should the request take? The answer may depend on cost, latency, modality, reasoning depth, regulatory location, or provider health. A common early mistake is to hardcode one model choice per application and then call the system “multi-model” because several vendor accounts exist in a spreadsheet. True routing means that model selection becomes explicit policy rather than scattered habit.

A travel summary, a retrieval condensation step, and a final executive memo do not necessarily deserve the same model. Nor do a vision classification step, a dense reasoning step, and a low-risk extraction step. Routing matters because it turns model heterogeneity from a maintenance burden into a design resource. Once the middle layer governs the choice, upper layers can request work in terms of need rather than vendor-specific endpoint names.

Failover is part of routing rather than merely an infrastructure afterthought. If the primary model is unavailable or unhealthy, the system must decide whether to substitute another model, defer the task, or degrade the output class. These are business decisions expressed through technical routes. LinkMind’s emphasis on failover and route policy is therefore not cosmetic. It reflects the practical truth that production agent systems cannot be built on the assumption that every provider is always available and equally appropriate.

Routing Strategy	What It Optimizes	Typical Trade-off
Failover	Business continuity during provider failure	Quality may degrade unless monitored carefully
Cost-aware routing	Lower unit economics for simple tasks	Task classification must be reliable
Parallel invocation	Higher confidence or broader evidence collection	Higher cost and merge complexity
Best-route selection	Adaptive optimization across latency, cost, or quality	Requires good telemetry and policy tuning
Pass-through compatibility	Easy adoption by existing clients	Can hide governance if overused as a crutch

5.2 Orchestration Is the Work Path, Not the Model Path

Routing chooses where a request goes. Orchestration chooses how work unfolds. A workflow may require retrieval, then classification, then ticketing, then review, then final communication. These steps can be represented explicitly in a workflow runtime such as Hermes Agent, or they can be partially implicit inside a single agent loop. The distinction matters because the more procedural and inspectable the process must be, the more orchestration deserves an explicit representation.

However, not every multi-step flow is a multi-agent system. A single agent can perform several steps if it retains one role and one responsibility boundary. Multi-agent design begins when roles diverge meaningfully. One agent may be allowed to research but not to act. Another may be allowed to act but not to approve. A third may be designed specifically to review output, policy citations, and trace. When these roles split, orchestration must handle handoffs as well as steps.

This is where Hermes Agent often finds its most natural role. It can express procedural graphs clearly. Yet the graph should not become the sole owner of shared skills, policy semantics, or provider logic. Those belong lower in the stack. Otherwise each workflow becomes a miniature platform, and consistency disappears.

5.3 Multi-Agent Design as Responsibility Architecture

Multi-agent design is easiest to misunderstand because it is easy to make it look sophisticated. Several named agents can exchange messages and still add no real value if they do not embody distinct responsibilities. A helpful test is to ask whether each agent could be given a separate performance review. If the answer is no, then the design may merely be fragmented prompting rather than meaningful role decomposition.

Meaningful roles often differ along four dimensions. They differ in objective: researcher, planner, operator, reviewer. They differ in tool rights: one can retrieve knowledge, another can write to systems, another can approve but not act. They differ in answer style or output obligation: one produces evidence notes, another produces machine actions, another produces executive summaries. And they differ in evaluation: each role succeeds by a different criterion. Once these differences exist, collaboration begins to make sense.

The deeper value of multi-agent decomposition is not that several models are speaking. It is that responsibility becomes local while governance remains shared. LinkMind is useful precisely because it allows role diversity above while preserving common routes, common knowledge, common capabilities, common safety, and common observability below.

5.4 The Topology of Multiple Lobsters, One LinkMind

The phrase “multiple Lobsters, one LinkMind” describes more than a networking pattern. It describes an operating model in which several interactive agents or operator workbenches share one control plane. Each Lobster instance may embody a different prompt, task focus, or department role. Yet all of them consume the same middle-layer provider endpoint, the same skill registry, the same safety fence, and the same routing policy. This dramatically reduces duplication and makes governance visible.

The same pattern extends naturally when Hermes Agent joins the environment. Hermes can act as a structured workflow or reviewer plane while Lobster remains the human-facing exploration plane. LinkMind sits beneath both. In this arrangement, Lobster does not need to know the details of vectorization, fallback, or capability blacklists. Hermes does not need to own every provider configuration or safety filter locally. Both become more useful precisely because they are no longer asked to be complete platforms.

This topology is also organizationally sane. Domain teams can own the prompts, interaction patterns, and workflow graphs that reflect their business practice. A platform team can own routing templates, capability registration, corpora boundaries, and audit conventions. Without this split, either the platform becomes overly controlling or the domains become infrastructure maintainers by accident.

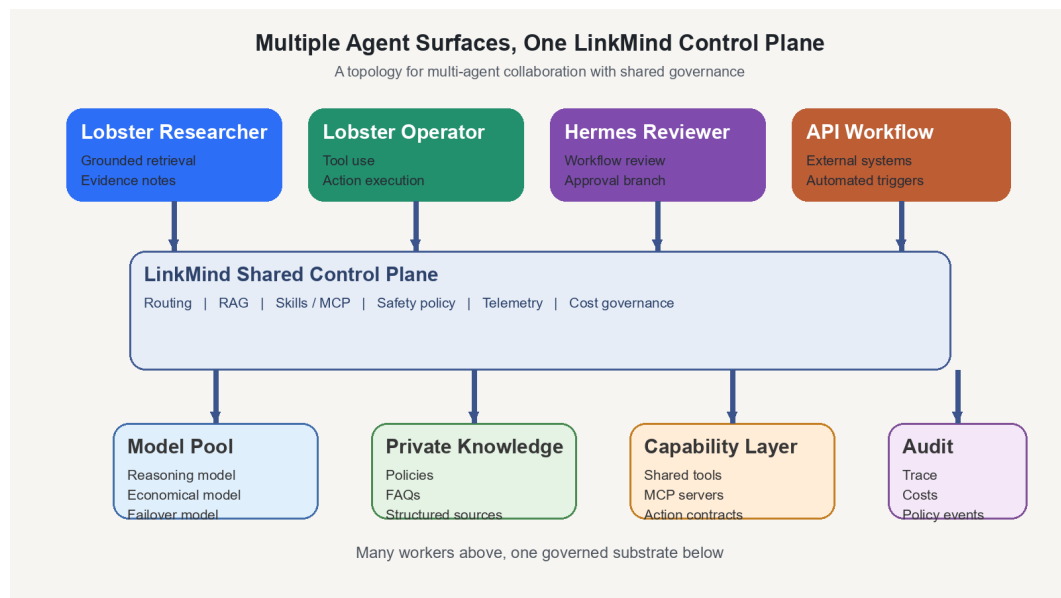


Figure 5.1. Multiple agent surfaces sharing one LinkMind control plane for routing, knowledge, capabilities, and audit.

5.5 Hands-On Lab: A Multi-Agent Incident Workflow

The lab in this chapter assembles a small multi-agent team around an incident response scenario. An employee reports a suspected phishing email on a company laptop. The system must determine the relevant policy, identify the asset owner, open the necessary service ticket, and draft a concise executive note. We use three roles. The Researcher retrieves policy and incident guidance. The Operator verifies asset facts and creates the ticket. The Reviewer checks policy alignment, trace quality, and the appropriateness of the final communication.

This scenario is useful because each role has a different success criterion. The Researcher must be grounded. The Operator must be precise and safe in tool use. The Reviewer must be skeptical, policy-aware, and concise. At the

same time, all three roles should share one LinkMind layer for model routes, capabilities, safety controls, and traces. The build therefore demonstrates both collaboration and common governance.

▷ **Step 1. Place every agent surface behind the same LinkMind endpoint.**

Repoint each Lobster instance and any Hermes workflow nodes from raw provider URLs to the common LinkMind endpoint. This is the architectural pivot on which the rest of the lab depends. Without it, the three roles remain separate clients with no common control surface. With it, they become differentiated consumers of one governed substrate.

The practical test is simple. Each role should still be able to converse or execute as before, but provider choice, capability exposure, and trace generation should now be visible centrally. If this step is incomplete, later evaluation of routing or policy will be misleading because the system will still be fragmented underneath.

▷ **Step 2. Register shared knowledge and capabilities once.**

- Researcher: access to phishing response policy, device handling procedure, and security FAQ corpora.
- Operator: access to `lookup_asset_owner` and `create_service_ticket`.
- Reviewer: access to traces, policy citations, and approval checkpoints, but not to unrestricted administrative tools.

The point of shared registration is not simply convenience. It ensures that when policy changes, tool semantics change, or a capability is disabled, all agents inherit the change coherently. This is the systems version of having one source of truth for behavior rather than many partial copies.

▷ **Step 3. Route by task type rather than by client habit.**

```
# illustrative routing intent
routes:
- when: task == "knowledge_retrieval_summary"
  use: economical-chat
- when: task == "final_executive_note"
  use: reasoning-chat
- when: task == "provider_error"
  fallback: backup-chat
```

The route plan is intentionally modest. Retrieval summarization may not need the most expensive reasoning model. Final synthesis for an executive note may justify a stronger backend. Provider errors should trigger a defined fallback. This structure teaches an important discipline: model strength should be allocated by task need, not by superstition or convenience.

Teams that route this way also gain clearer cost visibility. Token usage can be attributed not just to an application, but to a role and a task type. This makes later optimization far more rational than blanket efforts to “use fewer tokens.”

▷ **Step 4. Run the incident workflow end to end.**

- Researcher receives the phishing report and returns the relevant policy facts with citations.
- Operator confirms the affected asset and opens the service ticket using the shared capability layer.
- Reviewer checks whether the ticket summary, policy basis, and final executive note align and remain within role boundaries.

Observe whether the handoffs are semantically clean. The Researcher should not quietly perform operational actions. The Operator should not improvise policy beyond the retrieved evidence. The Reviewer should not become a second operator. Clear division of labor is what makes the system interpretable when several agents participate.

▷ **Step 5. Inspect collaboration as an operational system.**

- Can token use be attributed by agent role rather than only by application?
- Can the team see which model served which step and which fallback, if any, occurred?
- Can one shared capability be revised or disabled without editing all agents?
- If one provider fails, does the multi-agent workflow degrade gracefully rather than deadlock?
- Can an auditor reconstruct who decided, who acted, and which evidence supported the action?

This inspection step is what turns a pleasing demo into an operational pattern.

Collaboration is not only about whether the final note sounds good. It is about whether the organization can observe, price, govern, and evolve the system after deployment.

5.6 Observability, Cost, and Reliability in Teamed Agent Systems

Multi-agent systems create a temptation to focus only on emergent behavior while ignoring cost and reliability. In reality, shared control becomes more important as the agent count rises. A platform should be able to attribute token usage by department, project, and agent role; reveal route choices; show fallback frequency; and identify which capability calls dominate cost or latency. LinkMind's own positioning around unified observability and token accounting should therefore be read as a core engineering requirement rather than as a cosmetic dashboard feature.

Reliability also becomes a graph problem. A single unavailable provider may affect only one step, or it may stall the entire collaborative path if no fallback has been defined. Teams should therefore evaluate not only node reliability but workflow resilience. Can the Researcher still produce a reduced answer if one corpus is unavailable? Can the Operator defer action while preserving the evidence trail? Can the Reviewer distinguish between policy uncertainty and infrastructure degradation? These questions determine whether multi-agent design is genuinely robust or merely theatrical.

5.7 The Team Operating Model

The technical topology of multi-agent systems is inseparable from the human topology of the teams that maintain them. A useful operating model often divides ownership as follows. The platform team owns LinkMind configuration, route patterns, shared skills, capability policies, audit schema, and common corpora conventions. Domain teams own prompts, role definitions, workflow composition, evaluation cases, and user-facing interaction patterns. Security or compliance teams may co-own policy review, incident benchmarks, and high-risk capability approval.

This split matters because it keeps expertise where it belongs. Domain teams understand business tasks and should shape agent roles accordingly. Platform teams understand reliability, reuse, and governance, and should

shape the control plane accordingly. If domains own all infrastructure locally, inconsistency proliferates. If the platform owns every prompt and workflow detail, business fitness decays.

“Multiple Lobsters, one LinkMind” is therefore not only a deployment phrase. It is a governance principle. Many workers may exist. One shared control plane should still define the rules of engagement. The same principle holds if Hermes Agent is added: many role-specialized surfaces above, one governable substrate below.

5.8 Common Anti-Patterns

- Calling a system multi-agent when it merely contains several prompts with no real responsibility separation.
- Embedding route logic independently in every client instead of centralizing it.
- Letting workflow engines accumulate local copies of shared capability semantics.
- Ignoring cost attribution until after several agents are already in production.
- Designing collaboration without audit, making it impossible to reconstruct who did what.

5.9 Collaboration Patterns beyond the Simple Trio

The Researcher-Operator-Reviewer trio used in the lab is only one collaboration pattern. Several others recur in practice. A planner-executor-reviewer pattern separates task decomposition from action and from quality control. A supervisor-worker pattern uses one coordinating agent to assign work to specialized workers. A market or debate pattern lets several agents propose alternatives before a judge or merger step chooses among them. An event-driven pattern allows agents to react to system events rather than to a single conversational thread.

Each pattern distributes responsibility differently. The planner-executor-reviewer pattern is strong where correctness matters more than speed because it inserts structured critique. The supervisor-worker pattern is useful when tasks fan out across domains or data sources. Debate-style patterns

may improve robustness when several uncertain interpretations compete. Event-driven patterns are common in operations settings where agents must react to signals such as alerts, tickets, or document arrivals.

The important architectural point is that these patterns should still consume shared routes, capabilities, and safety policy. Otherwise each pattern becomes its own mini-platform. LinkMind is valuable here because it can keep the lower substrate stable while the upper collaboration pattern changes. Teams may discover that the best collaboration design for finance differs from the best design for incident response, yet both can remain governable if the shared control plane is intact.

Textbook maturity appears when teams stop arguing about which collaboration pattern is universally best and start matching patterns to risk, latency, domain shape, and human review needs.

5.10 Distributed Topologies: Proxy, GEO, Cascade, and Local Acceleration

As agent use grows across regions or departments, routing and governance often require a more distributed topology. The LinkMind materials refer to proxy nodes, geographically aware deployment, cascade layering, and local acceleration patterns such as Medusa. These concepts are important because they show that a control plane can itself be layered or regionalized without ceasing to be a control plane.

A proxy pattern is useful when several teams need one managed upstream exit to external models. It centralizes credentials, route rules, safety filters, and cost accounting. A GEO-aware pattern becomes important when latency, network policy, or regulatory location matters. A cascade pattern separates responsibilities across several LinkMind layers, such as one safety-focused layer and one routing-focused layer. A local acceleration layer becomes valuable in on-premise or edge settings where repeated prompts and limited GPU resources make caching economically meaningful.

These topologies should not be adopted simply because they sound advanced. Each one adds operational surface area. The question is whether the business need justifies the complexity. A large multi-region deployment may need them. A small internal pilot almost certainly does not. Good architecture grows by necessity, not by ambition alone.

What remains constant across these shapes is the principle of central policy with distributed execution. Even when several nodes exist, the organization still needs a coherent way to define route rules, safety boundaries, capability exposure, and observability conventions. Otherwise distribution becomes fragmentation.

5.11 Measuring Maturity in Multi-Agent Programs

Mature multi-agent programs are not defined by the sheer number of agents. They are defined by how well the organization can explain, observe, and evolve those agents. A weak program may have many named agents but no reliable ownership, cost visibility, safety consistency, or evaluation discipline. A strong program may have fewer agents but clear role boundaries, shared capabilities, stable routes, and a benchmark suite that reveals regression quickly.

One helpful maturity lens has four levels. At level one, teams build isolated demos around direct model calls. At level two, one or more agents become useful but still carry local copies of provider and tool logic. At level three, a shared control plane emerges, centralizing routes, safety, and common capabilities. At level four, the organization operates several roles, workflows, or regions atop that shared plane with measurable cost, reliability, and governance. The purpose of such a model is not bureaucracy. It is self-awareness.

Metrics should reflect this maturity. Early metrics may emphasize task completion and user delight. Later metrics must include route performance, fallback frequency, corpus freshness, tool failure rate, safety denial accuracy, cost per workflow, and review burden per agent role. Once the system becomes a platform, platform metrics become unavoidable.

Maturity models are useful when they encourage sequencing. They become harmful when they are used to shame teams into complexity before the foundations are ready. The goal is not to become multi-agent quickly. The goal is to become multi-agent coherently.

5.12 Adoption Roadmap and Organizational Design

A realistic adoption roadmap often starts with one or two narrow agents that prove value in a bounded workflow. The next stage introduces shared knowledge or shared capabilities so that duplication begins to fall. The third

stage moves route policy, safety policy, and observability into a central layer such as LinkMind. Only then does broad multi-agent collaboration usually become healthy, because the organization has a substrate strong enough to support role diversity without chaos.

Organizational design should follow the same rhythm. Early on, a single small team may own everything because the scope is modest. As adoption grows, specialization becomes useful. A platform group owns routes, capabilities, and safety infrastructure. Domain teams own role prompts, workflows, and evaluation cases. Security and compliance teams co-own high-risk capability release and incident review. The important thing is not rigid hierarchy. It is explicit ownership.

The user's phrase "multiple Lobsters, one LinkMind" captures this roadmap elegantly. Many interactive agents may bloom above. One common substrate should still govern how those agents reach models, corpora, tools, and policy. The same idea extends naturally when Hermes Agent or other orchestrators join the stack. Multiplicity above need not imply fragmentation below.

The classic mistake is to delay platform thinking until many agents already exist. By that time local habits have hardened and convergence becomes expensive. A textbook on agent engineering should therefore emphasize that platformization is not the enemy of experimentation. Done at the right moment, it is what allows experimentation to scale without dissolving into technical debt.

Chapter Summary

- Routing, orchestration, and multi-agent responsibility are distinct layers of design and should be evaluated separately.
- A shared control plane turns model diversity, capability reuse, and safety policy into platform assets rather than client burdens.
- Multiple Lobster agents can coexist effectively when LinkMind centralizes routes, skills, corpora, and audit.
- Hermes Agent complements rather than replaces such a setup when it is used for structured workflow or review.
- The organizational split between platform ownership and domain ownership is as important as the technical split.

Review Questions

- Why should routing policy live separately from workflow definition?
- What distinguishes a real multi-agent design from a fragmented single-agent one?
- How does the pattern of multiple Lobsters over one LinkMind improve governance?
- Why is cost attribution by role or task type important in multi-agent systems?
- How would you divide ownership between platform and domain teams in a scaled agent program?

Further Study: Shared Standards and Cross-Team Budgeting

Once several teams rely on a common agent substrate, governance and budgeting become inseparable from architecture. If a department uses a strong reasoning model for every step while another uses cheaper routes for preliminary work, should both costs be pooled or attributed separately? If several teams depend on the same capability or corpus, who funds improvements and who decides deprecation? These questions sound administrative, yet the platform must support them technically through routing visibility, token accounting, and dependency clarity.

Shared standards make these negotiations more rational. A common route naming scheme, common capability version conventions, common trace fields, and common benchmark practices reduce friction between teams. Standards do not eliminate local variation where it is useful. They prevent accidental variation where it is expensive. The result is that domain teams can remain creative in role and workflow design while the platform remains legible as a whole.

This is a moment where LinkMind's accounting and observability story becomes genuinely important. Cost attribution by department, project, or agent role is not just a dashboard feature. It is the basis for fair resource conversations. Without it, platform economics are governed by guesswork and politics rather than by evidence.

In mature organizations, such standards slowly become the constitution of

the agent platform. They encode shared expectations about how models are chosen, how capabilities are exposed, how safety is enforced, and how cost is understood. The platform becomes easier to scale because coordination friction falls.

Further Study: Migration from Isolated Successes to a Common Plane

Most organizations do not begin with elegant platform thinking. They begin with scattered successes: one Lobster assistant that operators love, one Hermes workflow that automates a painful review loop, one direct integration that answers internal questions tolerably well. Migration to a common plane is therefore a serious engineering topic. The aim is not to erase local success, but to converge the hidden infrastructure beneath that success.

A practical first step is often provider convergence. Existing clients are pointed to a common LinkMind endpoint so that token accounting, route policy, and baseline safety become shared without forcing immediate interface redesign. This is why the Agent Mate pattern is so useful. It lowers the social and technical cost of the first unification move. Only after traffic is flowing through a shared layer does it become sensible to standardize capabilities, corpora boundaries, and more ambitious orchestration patterns.

Migration should also be selective. Some legacy behaviors are valuable and deserve preservation. Others are accidents that became habits because they were never reviewed. Platformization is therefore an opportunity not only to combine systems, but to simplify them. The best migration programs remove unsafe or duplicative patterns while carrying forward the parts that genuinely help users.

When this process is handled well, teams feel less that centralization has stolen their autonomy and more that it has removed duplicated burden. Their successful local agents gain a stable home rather than being swallowed by a distant platform mandate. That psychological dimension is often the difference between a technically correct migration and a politically successful one.